

UNIT I

DEEP NETWORKS BASICS

Linear Algebra

Linear algebra is a fundamental **mathematical concept** that is essential for many machine learning algorithms and techniques.

1. Data Representation
2. Algorithmic Implementation
3. Optimization
4. Dimensionality Reduction
5. Efficiency

Fundamental Concepts in Linear Algebra

1. Scalars

Scalars are single numbers that scale vectors or matrices.
In linear algebra, a scalar is a **single number**.

Scalar
24

2. Vectors

In linear algebra, a vector is a **1-dimensional array**.
In Machine Learning, vectors are used to represent input data, weights in models, and predictions.

Vector
 $\begin{bmatrix} 2 & -8 & 7 \end{bmatrix}$
row
or
column $\begin{bmatrix} 2 \\ -8 \\ 7 \end{bmatrix}$

3. Matrices

In linear algebra, a matrix is a **2-dimensional array** arranged in rows and columns.

It is used to organize data in a tabular format, where rows represent different data points and columns represent features.

Matrix

$$\begin{bmatrix} 6 & 4 & 24 \\ 1 & -9 & 8 \end{bmatrix}$$

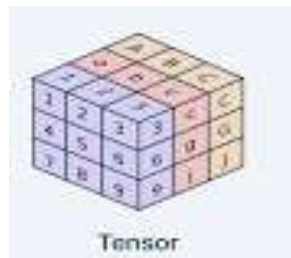
row(s) × column(s)

4. Tensors

A Tensor is an **N-dimensional Matrix**.

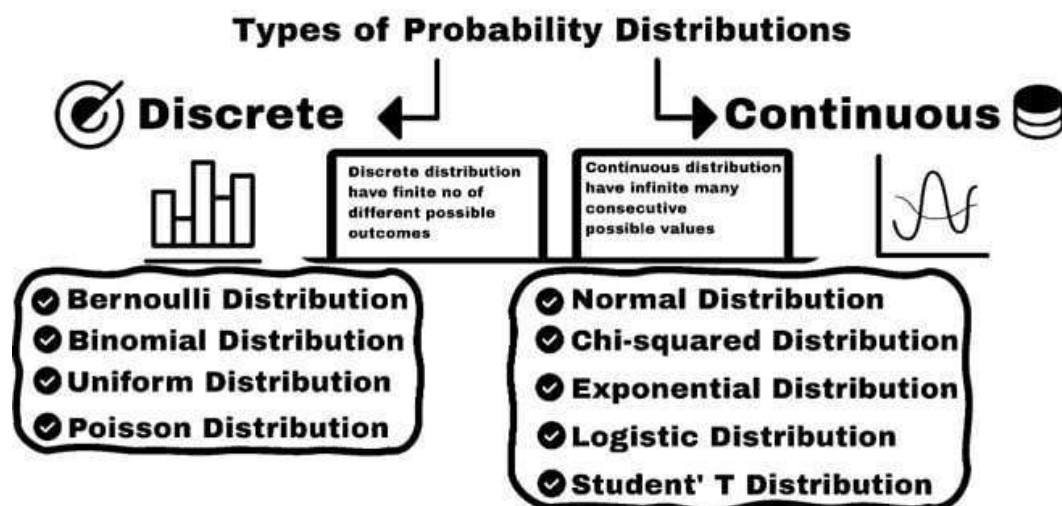
Tensor

$$\begin{bmatrix} [1 \ 2] & [3 \ 4] \\ [5 \ 6] & [7 \ 8] \\ [9 \ 0] & [1 \ 2] \end{bmatrix}$$



Probability Distribution

A probability distribution is a mathematical function that assigns the probabilities of different outcomes to the possible values of a random variable. It provides a way of modeling the likelihood of each outcome in a random experiment.



Bernoulli Distribution?

A Bernoulli distribution is a discrete probability distribution that models a random variable with only two possible outcomes. These outcomes are typically labeled as "success" and "failure," or else they are represented numerically as 1 and 0.

- Flipping a coin (heads or tails)
- Answering a true/false question
- Determining if a customer will make a purchase (buy or not buy)

$$P(X = 1) = p, \quad P(X = 0) = 1 - p$$

Binomial Distribution

Binomial Distribution is a probability distribution used to model the number of successes in a fixed number of independent trials, where each trial has only two possible outcomes: success or failure.

- Flipping a coin (heads or tails) 3 times

The Binomial Distribution Formula, which is used to calculate the probability, for a random variable $X = 0, 1, 2, 3, \dots, n$ is given as

$$P(X = r) = {}^nC_r p^r (1-p)^{n-r}, \quad r = 0, 1, 2, 3, \dots$$

Where,

- **n** = Total number of trials
- **r** = Number of successes
- **p** = Probability of success

Uniform Distribution

A Uniform Distribution is a type of probability distribution in which every outcome in a given range is equally likely to occur. That means there is no bias—no outcome is more likely than another within the specified set.

- It has two parameters, a and b: a = minimum and b = maximum. The distribution is written as U (a, b).
- **For example:** Rolling a fair six-sided die: Each number (1, 2, 3, 4, 5, or 6) has an equal chance of being rolled, with a probability of 1/6 for each outcome.

Poisson Distribution

The Poisson distribution is a discrete probability distribution that calculates the likelihood of a certain number of events occurring within a fixed interval of time, assuming the events occur independently.

Examples:

- Websites: Clicks or visitors per minute.
- Traffic: Cars passing a point on a highway in an hour.

$$P(X = r) = \frac{e^{-\lambda} \lambda^r}{r!}$$

Where,

- $P(X = r)$ is the Probability of observing k Events
- e is the Base of the Natural Logarithm (approximately 2.71828)
- λ is the Average Rate of Occurrence of Events
- r is the Number of Events that occur

Normal Distribution (Gaussian Distribution)

The normal distribution is one of the most commonly used continuous distributions. It has a bell-shaped curve that is symmetric around the mean. A normal distribution, also known as a bell curve or Gaussian distribution.

Key properties:

- The mean, median, and mode are all equal.
- The curve is symmetric about the mean.

Chi-Squared Distributions

The Chi-Squared distribution (also chi-square or χ^2 -distribution) represents the distribution of the sum of the squares of k independent standard normal random variables. If $Z_1, Z_2, \dots, Z_k$ are independent standard normal random variables, the Chi-Squared variable is:

$$X^2 = Z_1^2 + Z_2^2 + \dots + Z_k^2$$

Exponential Distribution

The Exponential Distribution is one of the most commonly used probability distributions in statistics and data science. It is widely used to model the time or space between events in a Poisson process. In simple terms, it describes how long you have to wait before something happens, like a bus arriving or a customer calling a help center.

Probability Density Function (PDF)

The probability density function of the exponential distribution is:

$$f(x; \lambda) = \lambda e^{-\lambda x}, \quad x \geq 0$$

Where:

- $\lambda > 0$ is the rate parameter (how often events occur)
- x is the time or distance until the next event

Logistic Distribution

The logistic distribution is a continuous probability distribution often used to model data with heavier tails (higher kurtosis) than the normal distribution, such as growth processes, technology adoption, and logistic regression models.

Standard Logistic Distribution ($\mu = 0, \sigma = 1$):

◦ CDF: $F(x) = \frac{1}{1 + e^{-x}}$

◦ PDF: $f(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$

Student's t -distribution

The t -distribution, also known as Student's t -distribution, is a way of describing data that follow a bell curve when plotted on a graph, with the greatest number of observations close to the mean and fewer observations in the tails.

It is a type of normal distribution used for smaller sample sizes, where the variance in the data is unknown.

Formula for the t-Score

The t-score (or t-statistic) quantifies how many estimated standard errors the sample mean ($\bar{x}$) is from the population mean (μ):

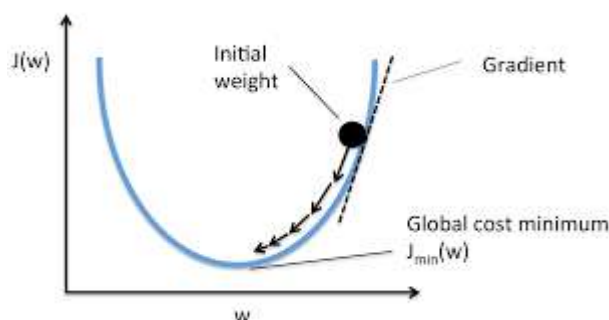
$$t = \frac{\bar{x} - \mu}{s/\sqrt{n}}$$

where,

- t = t-score,
- $\bar{x}$ = sample mean
- μ = population mean,
- s = standard deviation of the sample,
- n = sample size

Gradient Descent Algorithm

Gradient Descent is an optimisation algorithm used to minimize a model's error by iteratively adjusting its parameters. By moving step-by-step in the direction of the steepest decrease in the loss function, it helps machine learning models learn the best possible weights for better predictions.



Suppose you're at the top of a hill and your goal is to find the lowest point in the valley. You can't see the entire valley from the top, but you can feel the slope under your feet.

1. **Start at the Top:** You begin at the top of the hill (this is like starting with random guesses for the model's parameters).
2. **Feel the Slope:** You look around to find out which direction the ground is sloping down. This is like calculating the gradient, which tells you the steepest way downhill.

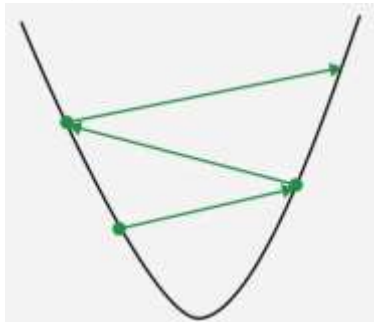
3. **Take a Step Down:** Move in the direction where the slope is steepest (this is adjusting the model's parameters). The bigger the slope, the bigger the step you take.
4. **Repeat:** You keep repeating the process feeling the slope and moving downhill until you reach the bottom of the valley (this is when the model has learned and minimized the error).

What is Learning Rate?

1. **If Learning rate is too small:** The algorithm will take tiny steps during iteration and converge very slowly. This can significantly increase training time and computational cost especially for large datasets.



2. **If Learning rate is too big:** The algorithm may take huge steps leading to overshooting the minimum of cost function without settling. It fails to converge, causing the algorithm to oscillate. This process is termed as exploding gradient problem.

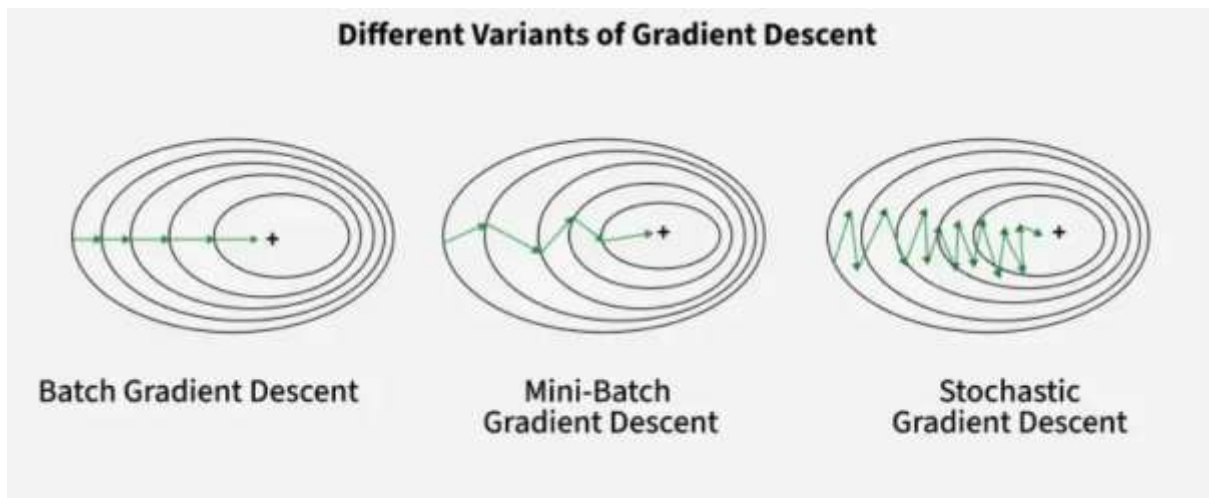


To address these problems we have some techniques that can be used:

- **Weights Regularizations:** The initialization of weights can be adjusted to ensure that they are in an appropriate range. Using a different activation function such as the Rectified Linear Unit (ReLU) can help us to mitigate the vanishing gradient problem.

- **Gradient clipping:** Restrict the gradients to a predefined range to prevent them from becoming excessively large or small.
- **Batch normalization:** It can also help to address these problems by normalizing the input of each layer to prevent activation function from saturating and hence reducing vanishing and exploding gradient problems.

Different Variants of Gradient Descent



1. Batch Gradient Descent

[Batch Gradient Descent](#) is a variant of the gradient descent algorithm where the entire dataset is used to compute the gradient of the loss function with respect to the parameters. In each iteration the algorithm calculates the average gradient of the loss function for all the training examples and updates the model parameters accordingly.

$$\theta = \theta - \eta \nabla J(\theta)$$

where:

- θ represents the parameters of the model
- η is the learning rate
- $\nabla J(\theta)$ is the gradient of the loss function $J(\theta)$ with respect to θ .

2. Stochastic Gradient Descent

[Stochastic Gradient Descent \(SGD\)](#) is a variant of the gradient descent algorithm where the model parameters are updated using the gradient of the

loss function with respect to a single training example at each iteration. Unlike batch gradient descent which uses the entire dataset SGD updates the parameters more frequently, leading to faster convergence.

$$\theta = \theta - \eta \nabla J(\theta; x^{(i)}, y^{(i)})$$

where:

- θ represents the parameters of the model,
- η is the learning rate,
- $\nabla J(\theta; x^{(i)}, y^{(i)})$ is the gradient of the loss function $J(\theta)$ with respect to θ for the i^{th} training example $(x^{(i)}, y^{(i)})$.

3. Mini-Batch Gradient Descent

[Mini-Batch Gradient Descent](#) is a compromise between Batch Gradient Descent and Stochastic Gradient Descent. Instead of using the entire dataset or a single training example Mini-Batch Gradient Descent updates the model parameters using a small, random subset of the training data called a mini-batch.

$$\theta = \theta - \eta \nabla J(\theta; \{x^{(i)}, y^{(i)}\}_{i=1}^m)$$

where:

- θ represents the parameters of the model,
- η is the learning rate,
- $\{x^{(i)}, y^{(i)}\}_{i=1}^m$ represents a mini-batch of m training examples,
- $\nabla J(\theta; \{x^{(i)}, y^{(i)}\}_{i=1}^m)$ is the gradient of the loss function $J(\theta)$ with respect to θ for the mini-batch.

Machine Learning Basics

Machine learning (ML) is a subfield of artificial intelligence that enables systems to learn from data, identify patterns, and make decisions or predictions without explicit programming. It improves performance through experience, using algorithms to analyze input data and enhance accuracy over time. Key types include supervised (labeled data), unsupervised (unlabeled data), and reinforcement learning (trial and error).

Capacity

Machine learning model capacity defines its ability to fit varied functions and learn complex patterns in data. It represents the range of functions a model

can approximate. High-capacity models (e.g., deep neural networks) can learn intricate patterns but risk overfitting, while low-capacity models (e.g., linear models) may underfit.

- **Hypothesis Space:** Capacity is determined by the size of the set of functions the model can choose from.
- **Overfitting (High Capacity):** The model learns the training data too well, including noise, leading to poor generalization on new data.
- **Underfitting (Low Capacity):** The model is too simple to learn the underlying structure of the data, resulting in high errors on both training and test sets.
- **Capacity Control:** Techniques to manage capacity include adjusting the number of parameters, layers, or applying regularization.

Underfitting and Overfitting

What is Underfitting?

Underfitting happens when the model fails to learn important patterns. It performs poorly on both training and testing data. Underfitting happens due to:

- Model is too simple
- Very high regularization
- Features are weak or missing
- Not enough training
- High bias

Bias: It is like assuming all birds can only be small and fly, so the model fails to recognize big birds like ostriches or penguins that can't fly and get biased with predictions.

Underfitting = High Bias + Low Variance

What is Overfitting?

Overfitting happens when the model learns too much from the training data, including noise and outliers. It performs very well on training data but poorly on test data. Overfitting happens due to:

- Model too complex
- Too many features
- Very little data
- No regularization
- High variance

Variance: Error that happens when a machine learning model learns too much from the data, including random noise.

Overfitting = Low Bias + High Variance

Bias

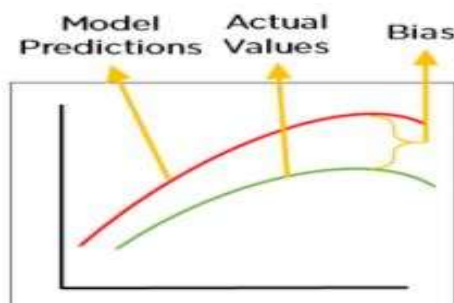
Bias is the error that occurs when a model is too simple to capture the true patterns in the data.

1. **High bias:** The model oversimplifies, misses patterns and underfits the data.
2. **Low bias:** The model captures patterns well and is closer to the true values

$$\text{Bias}^2 = (\mathbb{E}[\hat{f}(x)] - f(x))^2$$

Where,

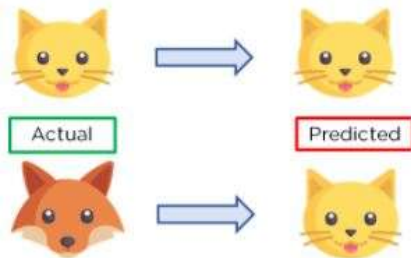
- $\hat{f}(x)$: predicted value by the model
- $f(x)$: true value
- $\mathbb{E}[\hat{f}(x)]$: expected prediction over different training sets



Variance

Variance arises when a model becomes too sensitive to training data and it captures noises in data too. It fails to give prediction on unseen new data.

1. **High variance:** The model is too sensitive to small changes and may overfit.
2. **Low variance:** The model is more stable but might miss some patterns.



$$\text{Variance} = \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right]$$

Where,

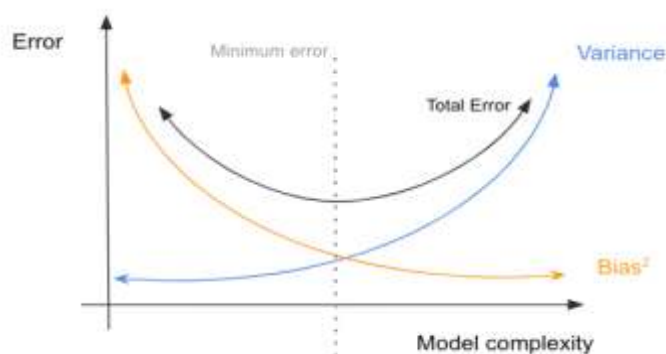
- $\hat{f}(x)$: predicted value by the model
- $\mathbb{E}[\hat{f}(x)]$: average prediction over multiple training sets

Bias-Variance Tradeoff

The relationship between bias and variance is often referred to as the [bias-variance tradeoff](#), which highlights the need for balance:

- Increasing model complexity reduces bias but increases variance (risk of overfitting).
- Simplifying the model reduces variance but increases bias (risk of underfitting).

The goal is to find an optimal balance where both bias and variance are minimized, resulting in good generalization performance.



Hyper parameters and validation sets

Hyperparameter Tuning

Most machine learning algorithms have several settings that we can use to control the behavior of the learning algorithm. These settings are called hyperparameters. It can have a significant impact on the performance of a model and can make the difference between a model that is accurate and robust and one that is unreliable and prone to errors.

There are several common hyperparameters that are tuned in machine learning models. These include:

Learning rate: the step size used to update the model weights during training.

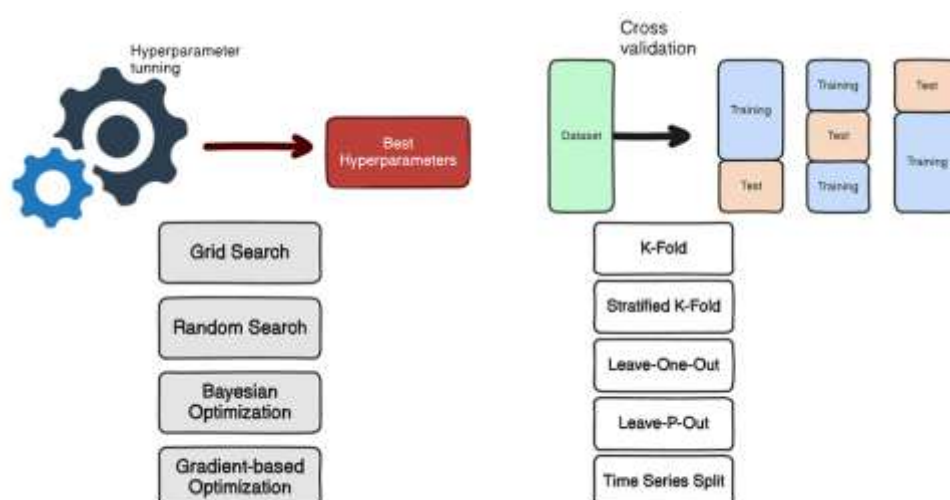
Batch size: the number of samples used in each batch during training.

Number of hidden layers: the number of layers in a deep learning model.

Number of neurons per layer: the number of neurons in each layer of a deep learning model.

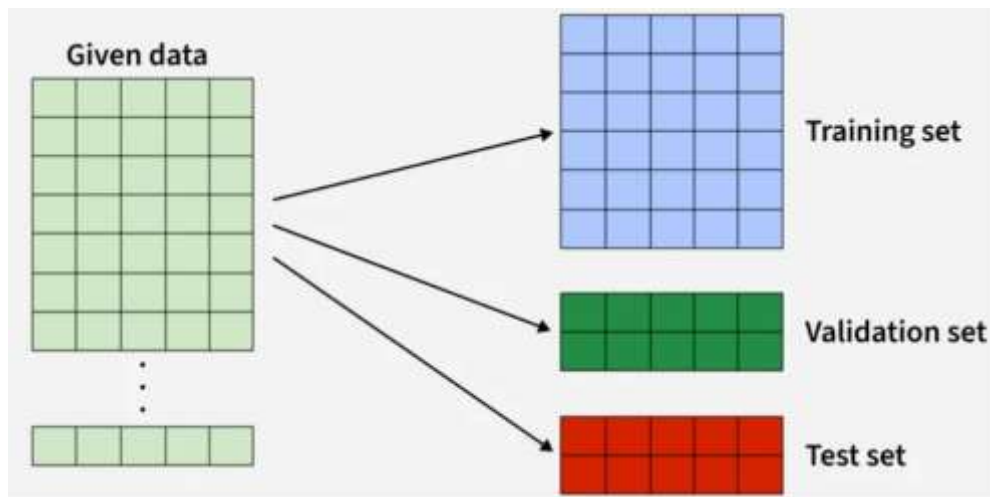
Regularization strength: the strength of the penalty applied to the model weights to prevent overfitting.

The optimal hyperparameters for a model will depend on the specific problem and data set. It is important to explore a range of hyperparameters and evaluate the performance of the model on a validation set to determine the best settings.



Training vs Testing vs Validation Sets

- **Training:** used to learn patterns
- **Validation:** used to tune and optimize the model
- **Testing:** used to measure final performance and generalization



1. Training Set

The training set is the portion of the dataset used to fit the machine learning model. During training, the algorithm learns patterns, relationships and parameters (such as weights in neural networks or coefficients in regression models) directly from this data. The model repeatedly adjusts itself to minimize the training error using optimization techniques like gradient descent.

- The model has full access to the labels and features in this dataset.
- Learning happens exclusively on the training set.
- Performance on training data does not reflect real-world performance.

2. Validation Set

The validation set is a separate subset of data used to tune model hyperparameters and make design decisions during training. Unlike the training set, it is not used to update model weights directly. Instead, it provides an unbiased estimate of model performance during development.

- Used during model selection and optimization
- Helps detect overfitting early
- Often involved in techniques like early stopping

3. Testing Set

The testing set is a completely independent subset used to evaluate the final model's performance after all training and tuning are complete. It simulates how the model will perform on unseen, real-world data and provides the most reliable estimate of generalization.

- Used only once (or very sparingly)
- No model decisions should be based on test results
- Represents real deployment conditions

Estimator

In machine learning, an estimator is an algorithm or function that learns patterns from data to make informed guesses (estimates) about unknown parameters or target variables, essentially forming the core of a model that can predict or classify new data, like a regression line or a decision tree. Good estimators are unbiased (average guess matches true value), consistent (gets closer with more data), and efficient (low variance). They train by finding optimal settings (parameters) using methods like [Maximum Likelihood Estimation](#) (MLE) to minimize errors, balancing underfitting and overfitting.

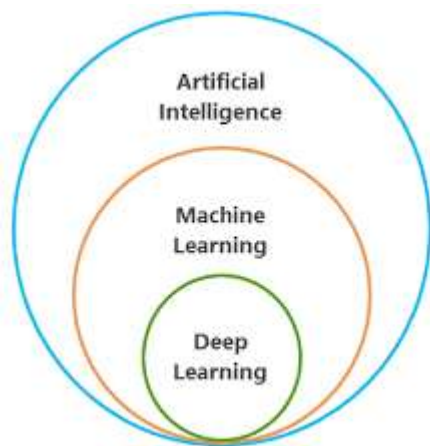
What they do?

- **Learn from data:** Take input features (X) and target values (y) to find relationships.
- **Estimate parameters:** Determine the best values for internal settings (weights, coefficients) that define the model.
- **Predict:** Use learned parameters to make predictions on new, unseen data.

Deep Learning

Deep learning is a subset of machine learning based on artificial neural networks with multiple layers (hence "deep") that mimic the human brain to analyze data.

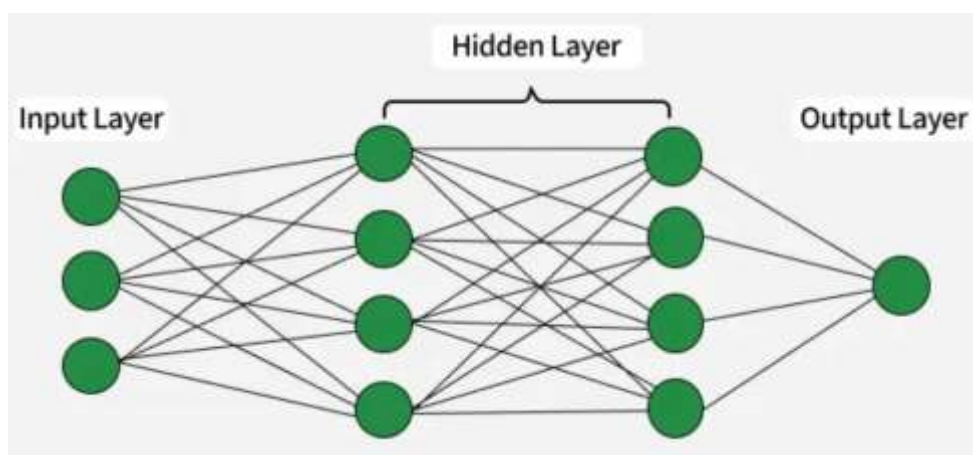
It automatically learns patterns from vast amounts of unstructured data—such as images, text, and audio—to perform complex tasks like facial recognition, language translation, and self-driving, often surpassing human accuracy.



How Deep Learning Works?

Neural network consists of layers of interconnected nodes or neurons that collaborate to process input data. In a fully connected deep neural network data flows through multiple layers where each neuron performs nonlinear transformations, allowing the model to learn intricate representations of the data.

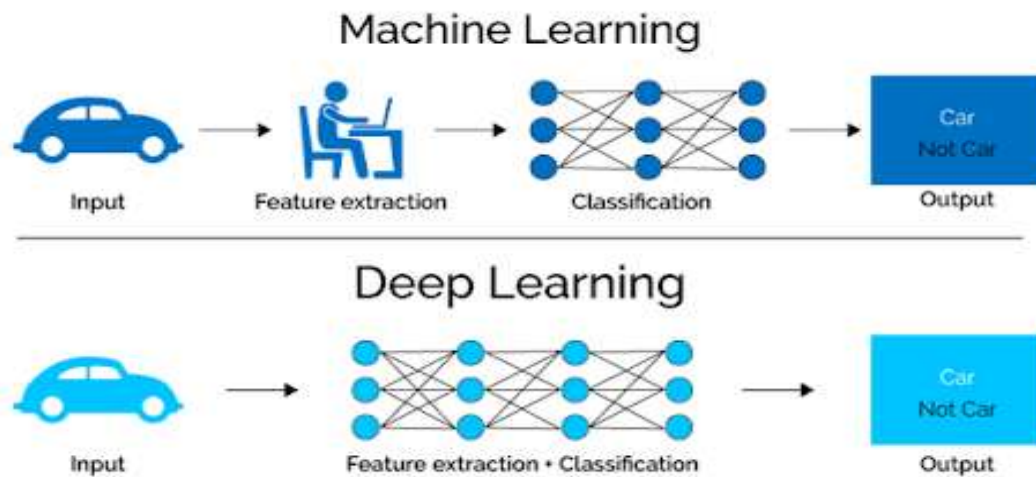
In a deep neural network the input layer receives data which passes through hidden layers that transform the data using nonlinear functions. The final output layer generates the model's prediction.



Difference between Machine Learning and Deep Learning

Machine learning and Deep Learning both are subsets of artificial intelligence but there are many similarities and differences between them.

Aspect	Machine Learning	Deep Learning
Basic Idea	Applies statistical algorithms to learn patterns from data	Uses artificial neural networks to learn patterns from data
Data Requirement	Works well with small to medium datasets	Requires a large amount of data
Task Complexity	Better for simple and low-label tasks	Better for complex tasks like image and text processing
Training Time	Takes less time to train	Takes more time to train
Feature Extraction	Features are manually selected and extracted	Features are automatically extracted
Learning Process	Not end-to-end	End-to-end learning
Model Complexity	Less complex	Highly complex
Interpretability	Easy to understand and explain	Hard to interpret (black box)
Hardware Requirement	Can run on CPU, needs less computing power	Needs GPU and high-performance systems
Use Cases	Spam detection, recommendation systems	Image recognition, NLP, speech recognition



Types of neural networks

1. [Feedforward neural networks \(FNNs\)](#): They are the simplest type of ANN, where data flows in one direction from input to output. It is used for basic tasks like classification.
2. [Convolutional Neural Networks \(CNNs\)](#): They are specialized for processing grid-like data, such as images. CNNs use convolutional layers to detect spatial hierarchies, making them ideal for computer vision tasks.
3. [Recurrent Neural Networks \(RNNs\)](#): They are able to process sequential data, such as time series and natural language. RNNs have loops to retain information over time, enabling applications like language modeling and speech recognition. Variants like LSTMs and GRUs address vanishing gradient issues.
4. [Generative Adversarial Networks \(GANs\)](#): This consists of two networks—a generator and a discriminator—that compete to create realistic data. GANs are widely used for image generation, style transfer and data augmentation.
5. [Autoencoders](#): They are unsupervised networks that learn efficient data encodings. They compress input data into a latent representation and reconstruct it, useful for dimensionality reduction and anomaly detection.
6. [Transformer Networks](#): It has revolutionized NLP with self-attention mechanisms. Transformers excel at tasks like translation, text generation and sentiment analysis, powering models like GPT and BERT.

Applications

1. Computer vision

In computer vision, deep learning models enable machines to identify and understand visual data. Some of the main applications of deep learning in computer vision include:

- **Object detection and recognition:** Deep learning models are used to identify and locate objects within images and videos, making it possible for machines to perform tasks such as self-driving cars, surveillance and robotics.
- **Image classification:** Deep learning models can be used to classify images into categories such as animals, plants and buildings. This is used in applications such as medical imaging, quality control and image retrieval.
- **Image segmentation:** Deep learning models can be used for image segmentation into different regions, making it possible to identify specific features within images.

2. Natural language processing (NLP)

In NLP, deep learning model enable machines to understand and generate human language. Some of the main applications of deep learning in NLP include:

- **Automatic Text Generation:** Deep learning model can learn the corpus of text and new text like summaries, essays can be automatically generated using these trained models.
- **Language translation:** Deep learning models can translate text from one language to another, making it possible to communicate with people from different linguistic backgrounds.
- **Sentiment analysis:** Deep learning models can analyze the sentiment of a piece of text, making it possible to determine whether the text is positive, negative or neutral.
- **Speech recognition:** Deep learning models can recognize and transcribe spoken words, making it possible to perform tasks such as speech-to-text conversion, voice search and voice-controlled devices.

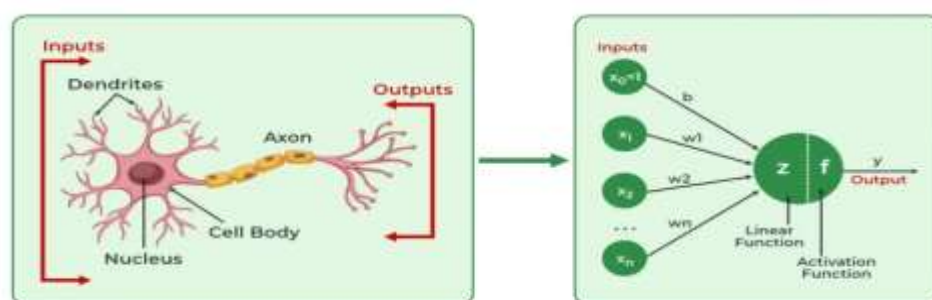
3. Reinforcement learning

In reinforcement learning, deep learning works as training agents to take action in an environment to maximize a reward.

- **Game playing:** Deep reinforcement learning models have been able to beat human experts at games such as Go, Chess and Atari.
- **Robotics:** Deep reinforcement learning models can be used to train robots to perform complex tasks such as grasping objects, navigation and manipulation.
- **Control systems:** Deep reinforcement learning models can be used to control complex systems such as power grids, traffic management and supply chain optimization.

Inspiration for artificial neural networks

A neuron is a special biological cell with information processing ability . It has a cell body and two outreaching tree-like branches: axon and dendrites. A neuron receives signals from other neurons through its dendrites, and transmits signals generated by its cell body to other neurons via its axon. A synapse is a place of contact between two neurons, an axon strand of one neuron and a dendrite strand of another neuron. A synapse can either enhance or inhibit the signal that passes through it. Learning occurs by changing the effectiveness of synapse. If the signals received exceeds a threshold, the neuron *fires*, i.e., it transmits a signal to other neurons. If not, it will not fire.



Layers in Neural Network Architecture

1. **Input Layer:** This is where the network receives its input data. Each input neuron in the layer corresponds to a feature in the input data.
2. **Hidden Layers:** These layers perform most of the computational heavy lifting. A neural network can have one or multiple hidden layers. Each

layer consists of units (neurons) that transform the inputs into something that the output layer can use.

3. **Output Layer:** The final layer produces the output of the model. The format of these outputs varies depending on the specific task like classification, regression.

How Feedforward Neural Networks Work

The working of a feedforward neural network involves two phases: the feedforward phase and the [backpropagation](#) phase.

- **Feedforward Phase:** In this phase, the input data is fed into the network, and it propagates forward through the network. At each hidden layer, the weighted sum of the inputs is calculated and passed through an activation function, which introduces non-linearity into the model. This process continues until the output layer is reached, and a prediction is made.

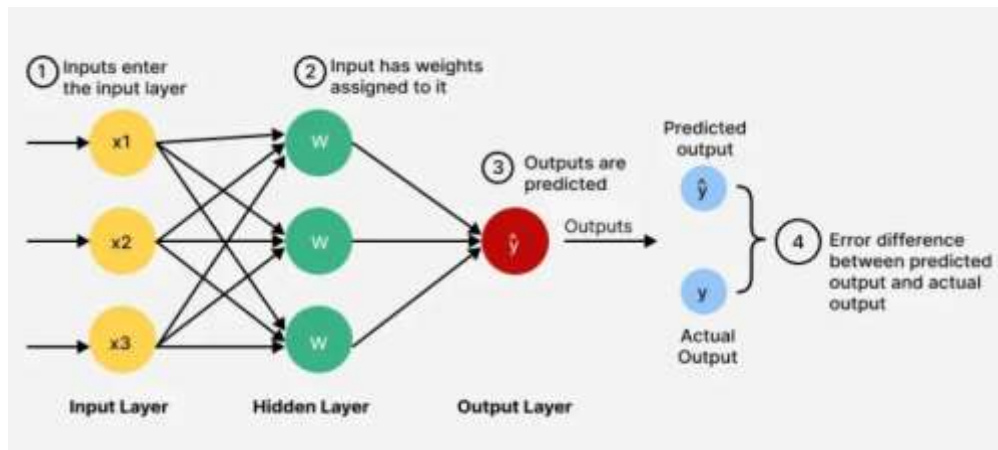
1. Linear Transformation: Each neuron in a layer receives inputs which are multiplied by the weights associated with the connections. These products are summed together and a bias is added to the sum. This can be represented mathematically as:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

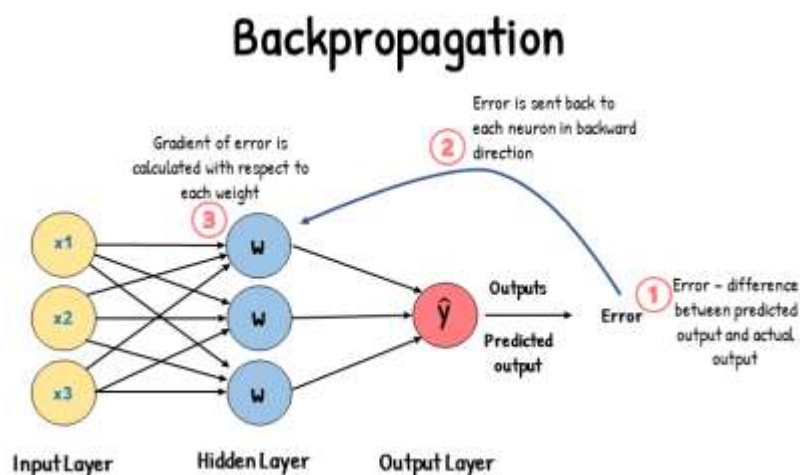
where

- w represents the weights
- x represents the inputs
- b is the bias

2. Activation: The result of the linear transformation (denoted as z) is then passed through an activation function. The activation function is crucial because it introduces non-linearity into the system, enabling the network to learn more complex patterns. Popular activation functions include ReLU, sigmoid and tanh.



- **Backpropagation Phase:** Once a prediction is made, the error (difference between the predicted output and the actual output) is calculated. This error is then propagated back through the network, and the weights are adjusted to minimize this error. The process of adjusting weights is typically done using a gradient descent optimization algorithm.



1. **Loss Calculation:** The network calculates the loss which provides a measure of error in the predictions. The loss function could vary; common choices are mean squared error for regression tasks or cross-entropy loss for classification.
2. **Gradient Calculation:** The network computes the gradients of the loss function with respect to each weight and bias in the network. This involves applying the chain rule of calculus to find out how much each part of the output error can be attributed to each weight and bias.

3. **Weight Update:** Once the gradients are calculated, the weights and biases are updated using an optimization algorithm like stochastic gradient descent (SGD). The weights are adjusted in the opposite direction of the gradient to minimize the loss. The size of the step taken in each update is determined by the learning rate.

3. Iteration

This process of forward propagation, loss calculation, backpropagation and weight update is repeated for many iterations over the dataset. Over time, this iterative process reduces the loss and the network's predictions become more accurate.

Activation Functions

Activation functions play a crucial role in feedforward neural networks. They introduce non-linear properties to the network, which allows the model to learn more complex patterns. Common activation functions include the sigmoid, tanh, and [ReLU \(Rectified Linear Unit\)](#).

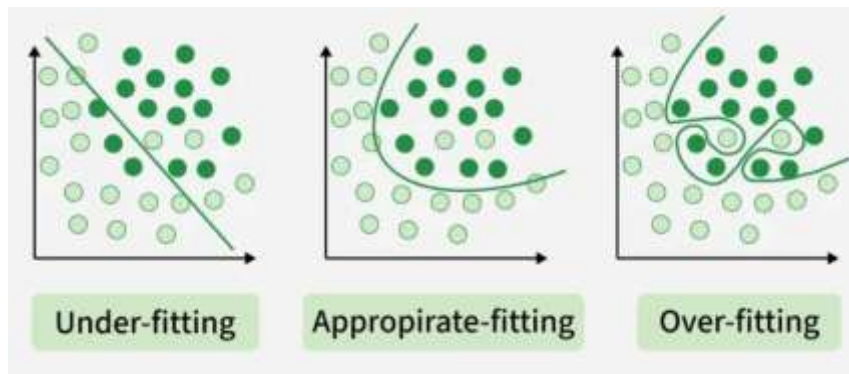
Training Feedforward Neural Networks

Training a feedforward neural network involves using a dataset to adjust the weights of the connections between neurons. This is done through an iterative process where the dataset is passed through the network multiple times, and each time, the weights are updated to reduce the error in prediction. This process is known as gradient descent, and it continues until the network performs satisfactorily on the training data.

Regularization

Regularization is a technique used in machine learning to prevent overfitting, which otherwise causes models to perform poorly on unseen data. By adding a penalty for complexity, regularization encourages simpler and more generalizable models.

- **Prevents overfitting:** Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- **Improves generalization:** Encourages simpler models that perform better on new, unseen data.



Types of Regularization

There are mainly 3 types of regularization techniques, each applying penalties in different ways to control model complexity and improve generalization.

1. Lasso Regression

A regression model which uses the L1 Regularization technique is called [LASSO \(Least Absolute Shrinkage and Selection Operator\)](#) regression. It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L). This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m |w_i|$$

Where

- m : Number of Features
- n : Number of Examples
- y_i : Actual Target Value
- $\hat{y}_i$: Predicted Target Value

2. Ridge Regression

A regression model that uses the L2 regularization technique is called [Ridge regression](#). It adds the squared magnitude of the coefficient as a penalty term to the loss function(L). It handles multicollinearity by shrinking the coefficients of correlated features instead of eliminating them.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$

Where,

- n : Number of examples or data points
- m : Number of features i.e predictor variables
- y_i : Actual target value for the i^{th} example
- $\hat{y}_i$: Predicted target value for the i^{th} example
- w_i : Coefficients of the features
- λ : Regularization parameter that controls the strength of regularization

3. Elastic Net Regression

[Elastic Net Regression](#) is a combination of both L1 as well as L2 regularization. That shows that we add the absolute norm of the weights as well as the squared measure of the weights. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization.

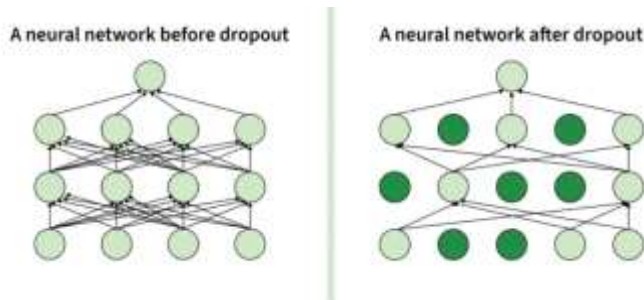
$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1 - \alpha) \sum_{i=1}^m |w_i| + \alpha \sum_{i=1}^m w_i^2 \right)$$

Where

- n : Number of examples (data points)
- m : Number of features (predictor variables)
- y_i : Actual target value for the i^{th} example
- $\hat{y}_i$: Predicted target value for the i^{th} example
- w_i : Coefficients of the features
- λ : Regularization parameter that controls the strength of regularization
- α : Mixing parameter where $0 \leq \alpha \leq 1$ and $\alpha = 1$ corresponds to Lasso (L_1) regularization, $\alpha = 0$ corresponds to Ridge (L_2) regularization and Values between 0 and 1 provide a balance of both L1 and L2 regularization

Dropout Regularization

Training deep learning models for too long on the same data can lead to overfitting where the model performs well on training data but poorly on unseen data. Dropout Regularization helps overcome this by randomly deactivating a portion of neurons during training hence forcing the model to learn more robust and independent features.



Early Stopping

Early stopping is a regularization technique that stops model training when overfitting signs appear. It prevents the model from performing well on the training set but underperforming on unseen data i.e validation set. Training stops when performance improves on the training set but degrades on the validation set, promoting better generalization while saving time and resources.

The technique monitors the model's performance on both the training and validation sets. If the validation performance worsens, training stops and the model retains the best weights from the period of optimal validation performance.



Choosing the Right Optimizer

Optimizer algorithms are essential for enhancing the performance of deep learning models by improving accuracy and training speed. They adjust the neural network's weights and learning rates during each training epoch to minimize the loss function.

- Optimizers adjust weights and learning rates in machine learning models.
- The choice of optimizer depends on the specific application.
- Randomly selecting optimizers can waste time with large datasets.

- A single epoch can be time-consuming when working with hundreds of gigabytes of data.

Gradient Descent Deep Learning Optimizer

1. SGD (Stochastic Gradient Descent) Optimizer

[Stochastic Gradient Descent \(SGD\)](#) updates the model parameters using the gradient of the loss function with respect to the weights. It is efficient, but can be slow, especially in complex models, due to noisy gradients and small updates.

```
import tensorflow as tf
```

```
model = tf.keras.Sequential([tf.keras.layers.Dense(1)])
```

```
# Compile the model with SGD optimizer
```

```
model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01,  
momentum=0.9), loss='mse')
```

```
model.fit(x_train, y_train)
```

2. Adam (Adaptive Moment Estimation)

[Adam](#) combines the advantages of two other extensions of SGD: AdaGrad and RMSProp.

It computes adaptive learning rates for each parameter by considering both first and second moments of the gradients. Adam is one of the most popular optimizers due to its efficient handling of sparse gradients and non-stationary objectives.

```
tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999,  
epsilon=1e-07)
```

Implementing Adam in Tensorflow using **tf.keras.optimizers()**:

```
import tensorflow as tf
```

```
model = tf.keras.Sequential([tf.keras.layers.Dense(1)])
```

```
# Compile the model with Adam optimizer
```

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
loss='mse')

model.fit(x_train, y_train)
```

3. RMSprop (Root Mean Square Propagation)

[RMSprop](#) is an adaptive learning rate method, that divides the learning rate by an exponentially decaying average of squared gradients. This optimizer is effective for handling non-stationary objectives and is often used for training RNNs.

```
tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9, epsilon=1e-07)
```

RMSprop can be implemented in TensorFlow using **tf.keras.optimizers.RMSprop()**:

```
import tensorflow as tf

model = tf.keras.Sequential([tf.keras.layers.Dense(1)])

# Compile the model with RMSprop optimizer

model.compile(optimizer=tf.keras.optimizers.RMSprop(learning_rate=0.001),
loss='mse')

model.fit(x_train, y_train)
```

4. Adagrad

[Adagrad](#) adapts the learning rate to the parameters by scaling it inversely with respect to the square root of the sum of all historical squared gradients. This helps in improving performance for sparse data. However, the learning rate tends to shrink too much over time, causing the optimizer to stop making updates.

```
tf.keras.optimizers.Adagrad(learning_rate=0.001, epsilon=1e-07)
```

Adagrad can be implemented in TensorFlow using **tf.keras.optimizers.Adagrad()**:

```
import tensorflow as tf

model = tf.keras.Sequential([tf.keras.layers.Dense(1)])
```

Compile the model with Adagrad optimizer

```
model.compile(optimizer=tf.keras.optimizers.Adagrad(learning_rate=0.01),  
loss='mse')
```

```
model.fit(x_train, y_train)
```

5. Adadelta

Adadelta is an extension of Adagrad. It addresses the problem of excessively diminishing learning rates. It uses a moving window of gradient updates, helping the model learn effectively even with sparse data.

```
tf.keras.optimizers.Adadelta(learning_rate=1.0, rho=0.95, epsilon=1e-07)
```

Adadelta can be implemented using **tf.keras.optimizer.Adadelta()**:

```
import tensorflow as tf
```

```
model = tf.keras.Sequential([tf.keras.layers.Dense(1)])
```

Compile the model with Adadelta optimizer

```
model.compile(optimizer=tf.keras.optimizers.Adadelta(learning_rate=1.0),  
loss='mse')
```

```
model.fit(x_train, y_train)
```

6. FTRL (Follow The Regularized Leader)

FTRL is an optimization algorithm particularly suited for problems with sparse data, such as those found in large-scale linear models. It maintains two accumulators to track gradients and updates them efficiently.

```
tf.keras.optimizers.Ftrl(learning_rate=0.1, learning_rate_power=-0.5,  
l1_regularization_strength=0.0, l2_regularization_strength=0.0)
```

Code Example of FTRL using **tf.keras.optimizers.Ftrl()**:

```
import tensorflow as tf
```

```
model = tf.keras.Sequential([tf.keras.layers.Dense(1)])
```

```
# Compile the model with FTRL optimizer
```

```
model.compile(optimizer=tf.keras.optimizers.Ftrl(learning_rate=0.01),  
loss='mse')
```

```
model.fit(x_train, y_train)
```

7. Nadam (Nesterov-accelerated Adaptive Moment Estimation)

Nadam combines Adam and Nesterov accelerated gradient. It calculates gradients using momentum and adapts the learning rate for each parameter, with an additional Nesterov momentum term.

```
tf.keras.optimizers.Nadam(learning_rate=0.001, beta_1=0.9, beta_2=0.999,  
epsilon=1e-07)
```

Nadam can be implemented using **tf.keras.optimizers.Nadam()**:

```
import tensorflow as tf
```

```
model = tf.keras.Sequential([tf.keras.layers.Dense(1)])
```

```
# Compile the model with Nadam optimizer
```

```
model.compile(optimizer=tf.keras.optimizers.Nadam(learning_rate=0.001),  
loss='mse')
```

```
model.fit(x_train, y_train)
```